Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

```sql
SELECT SUM(c.Population)
FROM CITY c
JOIN COUNTRY co
ON c.CountryCode = co.Code
WHERE co.Continent = 'Asia';
```

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the CONTINENT is 'Africa'.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

```sql
SELECT c.Name
FROM CITY c
JOIN COUNTRY co
ON c.CountryCode = co.Code
WHERE co.Continent = 'Africa';
```

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

```sql
SELECT co.Continent, FLOOR(AVG(c.Population)) AS avg_city_population
FROM CITY c
JOIN COUNTRY co
ON c.CountryCode = co.Code
GROUP BY co.Continent;
```

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

COUNTRY

Field	Type
CODE	VARCHAR2(3)
NAME	VARCHAR2(44)
CONTINENT	VARCHAR2(13)
REGION	VARCHAR2(25)
SURFACEAREA	NUMBER
INDEPYEAR	VARCHAR2(5)
POPULATION	NUMBER
LIFEEXPECTANCY	VARCHAR2(4)
GNP	NUMBER
GNPOLD	VARCHAR2(9)
LOCALNAME	VARCHAR2(44)
GOVERNMENTFORM	VARCHAR2(44)
HEADOFSTATE	VARCHAR2(32)
CAPITAL	VARCHAR2(4)
CODE2	VARCHAR2(2)

You are given two tables: Students and Grades. Students contains three columns ID, Name and Marks.

Column	Type
ID	Integer
Name	String
Marks	Integer

Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

```sql
SELECT
    CASE
        WHEN g.Grade < 8 THEN NULL
        ELSE s.Name
    END AS Name,
    g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
    ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE
        WHEN g.Grade >= 8 THEN s.Name
        ELSE NULL
    END ASC,
    CASE
        WHEN g.Grade < 8 THEN s.Marks
        ELSE NULL
    END ASC;
```

Ketty gives Eve a task to generate a report containing three columns: Name, Grade and Mark. Ketty doesn't want the NAMES of those students who received a grade lower than 8. The report must be in descending order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

Write a query to help Eve.

Sample Input

ID	Name	Marks
1	Julia	88
2	Samantha	68
3	Maria	99
4	Scarlet	78
5	Ashley	63
6	Jane	81

Sample Output

```
Maria 10 99
Jane 9 81
Julia 9 88
Scarlet 8 78
NULL 7 63
NULL 7 68
```

Sample Output

```
Maria 10 99
Jane 9 81
Julia 9 88
Scarlet 8 78
NULL 7 63
NULL 7 68
```

Note

Print "NULL" as the name if the grade is less than 8.

Explanation

Consider the following table with the grades assigned to the students:

ID	Name	Marks	Grade
1	Julia	88	9
2	Samantha	68	7
3	Maria	99	10
4	Scarlet	78	8
5	Ashley	63	7
6	Jane	81	9

So, the following students got 8, 9 or 10 grades:

- Maria (grade 10)
- Jane (grade 9)
- Julia (grade 9)
- Scarlet (grade 8)

https://www.hackerrank.com/challenges/full-score/problem?isFullScreen=true

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective hacker_id and name of hackers who achieved full scores for more than one challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending hacker_id.

Input Format

The following tables contain contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Difficulty: The difficult_level is the level of difficulty of the challenge, and score is the maximum score that can be achieved for a challenge at that difficulty level.

Column	Type
difficulty_level	Integer
score	Integer

```sql
SELECT
    h.hacker_id,
    h.name
FROM Hackers h
JOIN Submissions s
    ON h.hacker_id = s.hacker_id
JOIN Challenges c
    ON s.challenge_id = c.challenge_id
JOIN Difficulty d
    ON c.difficulty_level = d.difficulty_level
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(*) > 1
ORDER BY
    COUNT(*) DESC,
    h.hacker_id ASC;
```

Upload Code as File

Test against custom input

- Challenges: The challenge_id is the id of the challenge, the hacker_id is the id of the hacker who created the challenge, and difficulty_level is the level of difficulty of the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer
difficulty_level	Integer

- Submissions: The submission_id is the id of the submission, hacker_id is the id of the hacker who made the submission, challenge_id is the id of the challenge that the submission belongs to, and score is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

https://www.hackerrank.com/challenges/harry-potter-and-wands/problem?isFullScreen=true

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each non-evil wand of high power and age. Write a query to print the id, age, coins_needed, and power of the wands that Ron's interested in, sorted in order of descending power. If more than one wand has same power, sort the result in order of descending age.

Input Format

The following tables contain data on the wands in Ollivander's inventory:

- Wands: The id is the id of the wand, code is the code of the wand, coins_needed is the total number of gold galleons needed to buy the wand, and power denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

```sql
SELECT
    w.id,
    wp.age,
    w.coins_needed,
    w.power
FROM Wands w
JOIN Wands_Property wp
    ON w.code = wp.code
WHERE wp.is_evil = 0
AND w.coins_needed = (
    SELECT MIN(w2.coins_needed)
    FROM Wands w2
    JOIN Wands_Property wp2
        ON w2.code = wp2.code
    WHERE wp2.is_evil = 0
      AND w2.power = w.power
      AND wp2.age = wp.age
)
ORDER BY
    w.power DESC,
    wp.age DESC;
```

- Wands_Property: The code is the code of the wand, age is the age of the wand, and is_evil denotes whether the wand is good for the dark arts. If the value of is_evil is 0, it means that the wand is not evil. The mapping between code and age is one-one, meaning that if there are two pairs, $(code_1,\ age_1)$ and $(code_2,\ age_2)$, then $code_1 \neq code_2$ and $age_1 \neq age_2$.

Column	Type
code	Integer
age	Integer
is_evil	Integer

Julia asked her students to create some coding challenges. Write a query to print the hacker_id, name, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by hacker_id. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result.

Input Format

The following tables contain challenge data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Challenges: The challenge_id is the id of the challenge, and hacker_id is the id of the student who created the challenge.

Column	Type
challenge_id	Integer
hacker_id	Integer

```sql
WITH challenge_count AS (
    SELECT
        h.hacker_id,
        h.name,
        COUNT(c.challenge_id) AS total_challenges
    FROM Hackers h
    JOIN Challenges c
        ON h.hacker_id = c.hacker_id
    GROUP BY h.hacker_id, h.name
),
freq AS (
    SELECT
        total_challenges,
        COUNT(*) AS freq_count
    FROM challenge_count
    GROUP BY total_challenges
),
max_count AS (
    SELECT MAX(total_challenges) AS max_val
    FROM challenge_count
)

SELECT
    cc.hacker_id,
    cc.name,
    cc.total_challenges
FROM challenge_count cc
JOIN freq f
    ON cc.total_challenges = f.total_challenges
JOIN max_count m
WHERE
    f.freq_count = 1
    OR cc.total_challenges = m.max_val
ORDER BY
    cc.total_challenges DESC,
    cc.hacker_id ASC;
```

Upload Code as File

Test against custom input

https://www.hackerrank.com/challenges/contest-leaderboard/problem?isFullScreen=true

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the hacker_id, name, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending hacker_id. Exclude all hackers with a total score of 0 from your result.

Input Format

The following tables contain contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Submissions: The submission_id is the id of the submission, hacker_id is the id of the hacker who made the submission, challenge_id is the id of the challenge for which the submission belongs to, and score is the score of the submission.

Column	Type
submission_id	Integer
hacker_id	Integer
challenge_id	Integer
score	Integer

```sql
SELECT
    h.hacker_id,
    h.name,
    SUM(ms.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT
        hacker_id,
        challenge_id,
        MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) ms
ON h.hacker_id = ms.hacker_id
GROUP BY h.hacker_id, h.name
HAVING SUM(ms.max_score) > 0
ORDER BY
    total_score DESC,
    h.hacker_id ASC;
```

Upload Code as File

Test against custom input